Practical 1(e): Operators and Control Statements.

Terminal:

Steps:

1. Create the main folder `Mern Practicals`.
2. Open Command Prompt inside that folder.
3. Create a React project using `npm create vite@latest pract1`.
4. Select React.
5. Select JavaScript + React Compiler.
6. Select Oxlint.
7. Select Yes for "Install with npm and start now?"
8. Wait for installation to finish.
9. Open `http://localhost:5173/` in Chrome.

- Create a file named operators.js.
- Write the JavaScript code and save it.
- Open the terminal in the same folder.

OR USE THESE STEPS

-> **npm -v** /checks whether Node Package Manager is installed.

-> **node -v** /checks whether Node.js is installed.

mkdir Question1_Operators

C:\Users\HP\OneDrive\Desktop\MERN\Question1_Operators>

dir

node –version

node .\operators.js

operators.js

node operators.js

Practical 1(e): Operators and Control Statements

Code:

```
let x = 15;

let y = 10;


// Arithmetic Operators

console.log("Addition:", x + y);

console.log("Subtraction:", x - y);

console.log("Multiplication:", x * y);
```

```
console.log("Division:", x / y);

// Control Statement: if-else
if (x > y) {
   console.log("x is greater than y");
} else {
   console.log("y is greater than or equal to x");
}
  console.log("Saamiya Shah")
```

Q2. Display browser message using server response

Step 1: Create a new folder

1. **In VS Code Explorer, right-click the top-level MERN folder.**
2. **Select New Folder.**
3. **Name it: Question2_Express**

Step 2: Open the VS Code terminal

1. **Click Terminal in the top menu.**
2. **Select New Terminal.**
3. **Copy and paste these commands one at a time:**

Command 1: Enter your new folder

```
cd Question2_Express
```

Command 2: Create the Node.js project

```
npm init -y
```

Command 3: Install Express

```
npm install express
```

Create app.js

1. **In the left Explorer panel, right-click Question2_Express.**
2. **Click New File.**
3. **Type exactly: app.js**
4. **Run the server using:**

```
node app.js
```

Code:

```
const express = require("express");


const app = express();


const PORT = 3000;


app.get("/Saamiya", (req, res) => {
  res.send("Welcome to Node.js and Express!");
});


app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Q3. Print even numbers from 1–50.

Step 2: Create the JavaScript file

1. **Right-click Question3_EvenNumbers.**
2. **Select New File.**
3. **Name the file: even.js**
4. **Press Enter.**
5. **Your folder should look like:**

```
Question3_EvenNumbers
└── even.js
```

8. **Step 3: Enter the code**
9. Open `even.js` and paste this complete code:

Code:

```
console.log("Even numbers from 1 to 50:");


for (let i = 1; i <= 50; i++) {
  if (i % 2 === 0) {
```

```
    console.log(i);
  }
}
```

Save using Ctrl + S. Run the server using:

```
node even.js
```

Practical 4: Different Express routes

Step 1: Create app.js

1. **Expand Question4_ExpressRoutes in Explorer.**
2. **Right-click the folder.**
3. **Select New File.**
4. **Name it: app.js**

Step 2: Open the file

code:

```
const express = require("express");

const app = express();

const PORT = 3001;

// Home route
app.get("/", (req, res) => {
  res.send("Welcome to the Home Page!");
});

// About route
app.get("/about", (req, res) => {
  res.send("Welcome to the About Page!");
});
```

```
// Contact route
app.get("/contact", (req, res) => {
  res.send("Welcome to the Contact Page!");
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Save with Ctrl + S. Run the server using:

```
node app.js
```

Q5. Sum of two numbers using variables and function.

Step 1: Open App.jsx

In VS Code Explorer:

1. Expand Practical3.
2. Expand src.
3. Click App.jsx.

Step 2: Copy and paste this complete code

Delete the existing code in App.jsx and paste:

```
function App() {
  let num1 = 15;
  let num2 = 25;

  function add(a, b) {
    return a + b;
  }

  return (
    <>
```

```
        <h1>Practical 3C</h1>
        <h2>Name: Saamiya Shah</h2>
        <h2>First Number: {num1}</h2>
        <h2>Second Number: {num2}</h2>
        <h2>Sum = {add(num1, num2)}</h2>
      </>
    );
}

export default App;
```

Step 3: Save

Press Ctrl + S.

Terminal: Run the Development Server (if not running)

1. **Open the integrated terminal in VS Code (Ctrl + ~ or go to Terminal > New Terminal).**
2. **Make sure you are inside the practical-3 folder:** cd practical-3
3. **Start the Vite dev server:** npm run dev
4. **View and Verify the Output**
5. **Open your browser and navigate to:** http://localhost:5173

Q7. Apply basic CSS styling to a React component .

Step 1: Add Styling to App.css

1. **In the Explorer panel under practical-3/src/,click on App.css.**
2. **Replace its contents with this styling:**

Code

```
.container {
  text-align: center;
  color: blue;
  background-color: lightyellow;
  padding: 20px;
```

```
  border-radius: 10px;
  font-family: Arial, sans-serif;
  margin-top: 50px;
}
```

3. **Save the file (Ctrl + S).**

Step 2: Update App.jsx

1. Open **App.jsx** under practical-3/src/.
2. Replace all the code with the following:

JavaScript code:

```
import "./App.css";

function App() {
  return (
    <div className="container">
      <h1>Practical 3D</h1>
      <h2>Saamiya Shah</h2>
      <h1>Welcome to React</h1>
      <p>Learning CSS in React</p>
    </div>
  );
}

export default App;
```

Save the file (Ctrl + S).

Terminal: Run the Development Server (if not running)

1. **Open the integrated terminal in VS Code (Ctrl + ~ or go to Terminal > New Terminal).**
2. **Make sure you are inside the practical-3 folder:** cd practical-3
3. **Start the Vite dev server:** npm run dev
4. **View and Verify the Output**
5. **Open your browser and navigate to:** http://localhost:5173

Practical6: Send a JSON response from the server

. Where to put it:

1. **In your VS Code Explorer sidebar on the left, open the folder named** Question6_Express.
2. Click to open **app.js.**

The Code:

Replace all content inside app.js with this exact code:

JavaScript Code:

```
const express = require("express");
const app = express();
const PORT = 3000;

// Home Route
app.get("/", (req, res) => {
  res.send("Welcome to Express Server!");
});

// JSON API Route (Question 6)
app.get("/api/student", (req, res) => {
  res.json({
    rollNo: 101,
    name: "Saamiya",
    course: "BSc IT",
    semester: 5,
    marks: 85
  });
});
```

```
// Start Server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

3. How to run it:

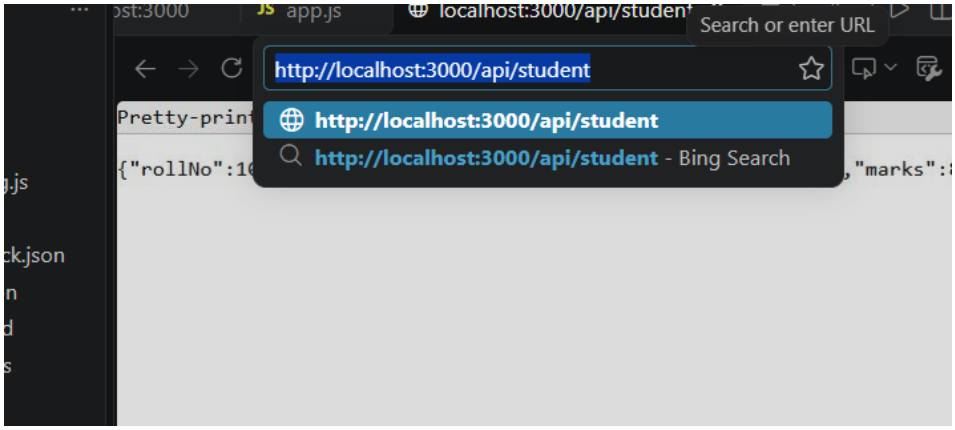
1. **Save the file (Ctrl + S).**

Terminal:

1. **In your terminal at the bottom, type:** node app.js
2. **Make sure it stays running and displays**: Server running at http://localhost:3000

Open Chrome or Edge and go to: Paste is as shown in screenshot

http://localhost:3000/api/student



Practical9:

Step 1: Create Component Files

1. In the Explorer on the left, expand ***practical-3 > src***.
2. Look for the components **folder.**
3. **Inside the components folder, create two new files:**
    - **Header.jsx**
    - **Footer.jsx**

Step 2: Add Code to Header.jsx

Open src/components/Header.jsx and paste:

JavaScript Code:

```
function Header() {
  return <h1>Welcome to MERN Stack Development</h1>;
}

export default Header;
```

(Press Ctrl + S to save)

Step 3: Add Code to Footer.jsx

Open src/components/Footer.jsx and paste:

JavaScript Code:

```
function Footer() {
  return <h3>Thank You for Visiting</h3>;
}

export default Footer;
```

(Press Ctrl + S to save)

Step 4: Update App.jsx

Open src/App.jsx and replace everything with:

JavaScript Code:

```
import Header from "./components/Header";
import Footer from "./components/Footer";

function App() {
  return (
    <div style={{ textAlign: "center", marginTop: "40px" }}>
      <h1>Practical 4A (Question 9)</h1>
      <h2>Saamiya Shah</h2>
      <Header />
      <p>This is the main content of the application.</p>
      <Footer />
    </div>
```

```
  );
}

export default App;
```

(Press Ctrl + S to save)

Step 5: View Output in Browser:

1. **Make sure your Vite server for practical-3 is running in your terminal:**

```
cd practical-3
npm run dev
```

2. **Open your browser to:**

Plaintext

http://localhost:5173

Question 10: "Fetch data from the API in a React application" (Practical 5D)

This practical uses React's useEffect and useState hooks to fetch external data from a public REST API (jsonplaceholder.typicode.com) and display it in a list. You can run this directly in your existing practical-3 project without needing MongoDB.

Step 1: Update src/App.jsx

1. Open **practical-3/src/App.jsx** in VS Code.
2. Replace its entire content with the exact standard code from your practical journal:

JavaScript CODE:

```
import React, { useState, useEffect } from 'react';


function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);


  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then((response) => response.json())
```

```
    .then((data) => {
      setUsers(data);
      setLoading(false);
    })
    .catch((error) => {
      console.error('Error fetching data:', error);
      setLoading(false);
    });
  }, []);

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif' }}>
      <h2>Practical 5D: Fetch Data from API</h2>
      {loading ? (
        <p>Loading user data...</p>
      ) : (
        <table border="1" cellPadding="8" style={{ borderCollapse: 'collapse', width: '100%'
}}>
          <thead>
            <tr style={{ backgroundColor: '#f2f2f2', textAlign: 'left' }}>
              <th>ID</th>
              <th>Name</th>
              <th>Email</th>
              <th>City</th>
            </tr>
          </thead>
          <tbody>
            {users.map((user) => (
              <tr key={user.id}>
                <td>{user.id}</td>
                <td>{user.name}</td>
```

```
            <td>{user.email}</td>
            <td>{user.address.city}</td>
          </tr>
        ))}
      </tbody>
    </table>
  )}
 </div>
 );
}

export default App;
```

Step 2: Save and View Output

1. Press **Ctrl + S** to save App.jsx.

Terminal: Run the Development Server (if not running)

1. **Open the integrated terminal in VS Code (Ctrl + ~ or go to Terminal > New Terminal).**
2. **Make sure you are inside the practical-3 folder:** cd practical-3
3. **Start the Vite dev server:** npm run dev
4. **View and Verify the Output**
5. **Open your browser and navigate to:** http://localhost:5173

Question 11: "React components - Pass data between components using props" (Practical 4B)

Step 1: Create the Child Component

1. **In the Explorer on the left, navigate to practical-3/src/components.**
2. **Create a new file named: Student.jsx**

Ensure Student.jsx Accepts Props

Click to open src/components/Student.jsx from your sidebar and make sure it has this code to display the props:

JavaScript Code: (Student.jsx)

```
function Student(props) {
```

```
 return (
  <div
   style={{
    border: "2px solid #333",
    borderRadius: "8px",
    padding: "16px",
    margin: "15px auto",
    maxWidth: "320px",
    textAlign: "left",
    backgroundColor: "#f9f9f9",
    color: "#111"
   }}
  >
   <p><strong>Roll No:</strong> {props.rollNo}</p>
   <p><strong>Name:</strong> {props.name}</p>
   <p><strong>Course:</strong> {props.course}</p>
   <p><strong>Semester:</strong> {props.semester}</p>
  </div>
 );
}

export default Student;
```

Step 3: Save and Check

1. **Press Ctrl + S to save both files.**
2. **JavaScript Code: (App.jsx)**

```
import Student from "./components/Student";

function App() {
 return (
  <div style={{ textAlign: "center", marginTop: "30px", fontFamily: "Arial, sans-serif" }}>
   <h1>Practical 4B (Question 11)</h1>
```

```
    <h2>Passing Data via Props</h2>

    {/* Passing data to Student component using props */}
    <Student rollNo={101} name="Saamiya" course="BSc IT" semester="Semester 5" />
    <Student rollNo={102} name="Ayesha" course="BSc IT" semester="Semester 5" />
    <Student rollNo={103} name="Nitin" course="BSc IT" semester="Semester 5" />
   </div>
  );
}

export default App;
```

Terminal: Run the Development Server (if not running)

1. **Open the integrated terminal in VS Code (Ctrl + ~ or go to Terminal > New Terminal).**
2. **Make sure you are inside the practical-3 folder:** cd practical-3
3. **Start the Vite dev server:** npm run dev
4. **View and Verify the Output**
5. **Open your browser and navigate to:** http://localhost:5173

Question 13: "Build a To-do list interface using React" (Practical 4F)

This practical uses React hooks (useState) to manage dynamic state for adding, checking, and deleting tasks. You can implement it directly in your existing practical-3 project.

Step 1: Update src/App.jsx

1. **Open practical-3/src/App.jsx in your editor.**

PNG

2. **Replace all the code inside with this complete To-do list implementation:**

JavaScript Code:

```
import { useState } from "react";
```

```
function App() {
  const [tasks, setTasks] = useState([
    { id: 1, text: "Complete Practical 6 (JSON Response)", completed: true },
    { id: 2, text: "Build React Components (Question 9)", completed: true },
    { id: 3, text: "Study MongoDB Operations", completed: false }
  ]);
  const [input, setInput] = useState("");

  const addTask = (e) => {
    e.preventDefault();
    if (!input.trim()) return;
    const newTask = {
      id: Date.now(),
      text: input,
      completed: false
    };
    setTasks([...tasks, newTask]);
    setInput("");
  };

  const toggleTask = (id) => {
    setTasks(
      tasks.map((task) =>
        task.id === id ? { ...task, completed: !task.completed } : task
      )
    );
};

  const deleteTask = (id) => {
    setTasks(tasks.filter((task) => task.id !== id));
```

```
  };

  return (
    <div
      style={{
        maxWidth: "420px",
        margin: "40px auto",
        padding: "24px",
        borderRadius: "10px",
        boxShadow: "0 4px 12px rgba(0,0,0,0.15)",
        fontFamily: "Arial, sans-serif",
        backgroundColor: "#ffffff",
        color: "#222"
      }}
    >
      <h2 style={{ textAlign: "center", marginBottom: "8px" }}>To-Do List</h2>
      <p style={{ textAlign: "center", color: "#666", fontSize: "14px", marginTop: 0 }}>
        Practical 4F (Question 13)
      </p>

      {/* Input Form */}
      <form onSubmit={addTask} style={{ display: "flex", gap: "8px", marginBottom: "20px"
}}>
        <input
          type="text"
          value={input}
          onChange={(e) => setInput(e.target.value)}
          placeholder="Enter a new task..."
          style={{
            flex: 1,
            padding: "8px 12px",
```

```
    border: "1px solid #ccc",
    borderRadius: "4px",
    fontSize: "14px"
   }}
  />
  <button
   type="submit"
   style={{
    padding: "8px 16px",
    backgroundColor: "#2563eb",
    color: "#fff",
    border: "none",
    borderRadius: "4px",
    cursor: "pointer",
    fontWeight: "bold"
   }}
  >
   Add
  </button>
</form>

{/* Task List */}
<ul style={{ listStyle: "none", padding: 0, margin: 0 }}>
 {tasks.map((task) => (
  <li
   key={task.id}
   style={{
    display: "flex",
    alignItems: "center",
    justifyContent: "space-between",
    padding: "10px 8px",
```

```
      borderBottom: "1px solid #eee"
    }}
   >
    <span
     onClick={() => toggleTask(task.id)}
     style={{
      textDecoration: task.completed ? "line-through" : "none",
      color: task.completed ? "#999" : "#333",
      cursor: "pointer",
      flex: 1
     }}
    >
     {task.text}
    </span>
    <button
     onClick={() => deleteTask(task.id)}
     style={{
      backgroundColor: "#ef4444",
      color: "#fff",
      border: "none",
      borderRadius: "4px",
      padding: "4px 8px",
      cursor: "pointer",
      fontSize: "12px"
     }}
    >
     Delete
    </button>
   </li>
  ))}
 </ul>
```

```
  </div>
 );
}

export default App;
```

Step 2: Save and View Output

1. **Press Ctrl + S to save App.jsx.**
2. **Look at your browser preview tab (or go to http://localhost:5173 / http://localhost:5176).**

Question 14: "Create different routes in Express" (Practical 6C)

This practical uses Express to create a server handling multiple URL routes (/, /about, /contact, /api/users) without requiring MongoDB.

Step 1: Create the Project Folder & File

1. **In your VS Code terminal, create and navigate to a new folder:**

Terminal:

```
cd C:\Users\HP\OneDrive\Desktop\MERN
mkdir Question14_Routes
cd Question14_Routes
npm init -y
npm install express
```

2.In the Question14_Routes folder, create a file named server.js.

Step 2: Add Code to server.js

Open server.js and paste the exact code from your submission manual:

JavaScript Code:

```
const express = require('express');
const app = express();
const PORT = 3000;
```

```
// Home Route
app.get('/', (req, res) => {
  res.send('Welcome to the Home Page!');
});

// About Route
app.get('/about', (req, res) => {
  res.send('About Us: This is a simple Express Routing application.');
});

// Contact Route
app.get('/contact', (req, res) => {
  res.send('Contact Us: reach out at contact@example.com');
});

// User API Route
app.get('/api/users', (req, res) => {
  res.json([
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
    { id: 3, name: 'Charlie' }
  ]);
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Step 3: Run the Server

In your terminal, execute:

```
node server.js
```

Step 4: Verify the Routes in Browser

Open your browser and navigate through the routes to take your verification screenshots:

- **Home:** http://localhost:3000/ displays *"Welcome to the Home Page!"*

Question 15: "React Routing - Create multiple pages and implement navigation between them" (Practical 5B/5C)

This practical uses react-router-dom to implement client-side navigation between different components/pages without page reload.

Step 1: Install React Router

Open your terminal in VS Code, make sure you are in the practical-3 folder, and run:

PowerShell

cd C:\Users\HP\OneDrive\Desktop\MERN\practical-3

npm install react-router-dom

Step 2: Create Page Components

Notice you already have a **Pages** folder inside src. Inside src/Pages, create three simple files:

1. **src/Pages/Home.jsx**:

JavaScript Code:

```
function Home() {
  return (
    <div style={{ textAlign: "center", marginTop: "20px" }}>
      <h2>Home Page</h2>
      <p>Welcome to our React Router Application!</p>
    </div>
  );
}

export default Home;
```

src/Pages/About.jsx:

JavaScript Code:

```
function About() {
  return (
    <div style={{ textAlign: "center", marginTop: "20px" }}>
      <h2>About Page</h2>
      <p>This page demonstrates client-side navigation using React Router.</p>
    </div>
  );
}

export default About;
```

3.src/Pages/Contact.jsx:

PNG

JavaScript Code:

```
function Contact() {
  return (
    <div style={{ textAlign: "center", marginTop: "20px" }}>
      <h2>Contact Page</h2>
      <p>Get in touch with us at contact@example.com</p>
    </div>
  );
}

export default Contact;
```

PNG

Step 3: Configure Routes in src/App.jsx

Code:

```
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";
import Home from "./Pages/Home";
import About from "./Pages/About";
import Contact from "./Pages/Contact";

function App() {
  const navStyle = {
    display: "flex",
    justifyContent: "center",
    gap: "20px",
    background: "#333",
    padding: "12px",
    listStyle: "none",
    margin: 0
  };

  const linkStyle = {
    color: "#fff",
    textDecoration: "none",
    fontWeight: "bold",
    fontSize: "16px"
  };

  return (
    <Router>
      <div style={{ fontFamily: "Arial, sans-serif" }}>
        {/* Navigation Bar */}
        <nav>
          <ul style={navStyle}>
            <li><Link to="/" style={linkStyle}>Home</Link></li>
            <li><Link to="/about" style={linkStyle}>About</Link></li>
```

```
          <li><Link to="/contact" style={linkStyle}>Contact</Link></li>
        </ul>
      </nav>

      {/* Route Definitions */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </div>
  </Router>
 );
}

export default App;
```

Step 4: Run & Verify

1. I**f the dev server is stopped, restart it in the terminal:**

```
npm run dev
```